

Camada de Transporte

TCP, Transferência confiável, Controle de Fluxo

Profa. Anelise Munaretto e Prof. Mauro Fonseca

DAINF - UTFPR

Material de Referência Bibliográfica

Kurose, J. F. & Ross, K. W. — *Redes de Computadores e a Internet: Uma Abordagem Top-Down*
https://gaia.cs.umass.edu/kurose_ross/

Como Funciona o TCP: Características e Segmento

Como Medir o Tempo: RTT, Timeout e Confiabilidade

Como Não Sobrecarregar o Receptor: Controle de Fluxo

Como Estabelecer Conversas: Gerenciamento de Conexão

Como Funciona o TCP: Características e Segmento

TCP: Visão Geral das Características

O TCP é o protocolo de transporte fundamental para aplicações que exigem entrega garantida acima da velocidade bruta

- **Ponto a Ponto:**

- Comunicação estrita entre um transmissor e um receptor (não suporta *multicast* ou *broadcast*)

- **Fluxo de Bytes Confiável e em Ordem:**

- Não existem “limites de mensagem”; a aplicação enxerga um fluxo contínuo de dados

- **Pipelining (Envio em Dutos):**

- Permite o envio de múltiplos segmentos em trânsito, limitados pelas janelas de fluxo e congestionamento

- **Full Duplex e Buffers:**

- Fluxo de dados bidirecional simultâneo na mesma conexão
- **MSS (Maximum Segment Size):** Tamanho máximo dos **dados** da aplicação (exclui cabeçalhos TCP/IP)

- **Orientado a Conexão:**

- Exige *Handshake* (3-way) para inicializar buffers e variáveis de estado antes da troca de dados
- O estado da conexão é mantido **apenas nos sistemas finais**, não nos roteadores

TCP: Controle e Limitação

O TCP não apenas entrega dados, mas atua ativamente para preservar a saúde da comunicação e da rede

Controle de Fluxo (Protege o Receptor)

Garante que o remetente não sobrecarregue os buffers de recepção do destino

- Baseado na **Janela de Recepção (rwnd)**, informada pelo receptor a cada segmento
- **Escopo:** Fim a fim (Remetente ↔ Receptor)

Controle de Congestionamento (Protege a Rede)

Limita a taxa de envio para evitar o colapso dos roteadores intermediários

- Baseado na **Janela de Congestionamento (cwnd)**, inferida pelo remetente (ex: perda de pacotes)
- **Escopo:** Remetente ↔ Rede

Janela Efetiva de Transmissão:

O remetente sempre respeita o limite imposto pela fórmula: $\min(rwnd, cwnd)$

Estrutura do Segmento TCP

32 bits (4 bytes)			
Porta de Origem (16 bits)		Porta de Destino (16 bits)	
Número de Sequência (32 bits)			
Número de Reconhecimento - ACK (32 bits)			
Head len	Reservado	U A P R S F	Janela de Recepção - <i>rwnd</i> (16 bits)
Checksum (16 bits)		Ponteiro de Dados Urgentes (16 bits)	
Opções (Tamanho variável - 0 a 40 bytes)			
...			
Dados da Aplicação (<i>Payload</i>)			
...			

Tamanho do Cabeçalho:

- **Tamanho Base:** 20 bytes (sem opções)
- **Head len (Data Offset):** Indica o tamanho do cabeçalho em **palavras de 32 bits**. (Ex: valor 5 significa $5 \times 4 = 20$ bytes)

Campos de Controle (Flags Principais):

- **SYN / FIN:** Estabelecimento e encerramento da conexão
- **ACK:** Indica que o campo de reconhecimento é válido
- **RST (Reset):** Rejeição de segmento ou aborto forçado da conexão

Como Medir o Tempo: RTT, Timeout e Confiabilidade

Números de Sequência e ACKs

A confiabilidade do TCP baseia-se na numeração precisa do fluxo de dados (orientado a **bytes**, não a mensagens)

Números de Sequência (Seq)

- Representam o número do **primeiro byte** de dados (*payload*) do segmento
- **ISN (Initial Seq Number):**
Escolhido de forma aleatória no *handshake* por segurança

Acknowledgements (ACKs)

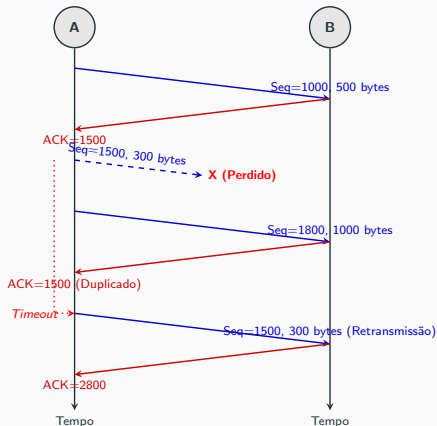
- Indicam o número de sequência do **próximo byte esperado** pelo receptor
- **Cumulativo:** Confirma todos os bytes recebidos em ordem até aquele ponto

Exemplo Prático e Dados Fora de Ordem

- **Dinâmica:** Host A envia $Seq = 0$ com 1000 bytes de dados. Host B responde com $ACK = 1000$
- **Fora de Ordem:** A RFC base não dita regras (descartar ou fazer *buffer*). Hoje, as pilhas TCP fazem *buffer* e utilizam a opção **SACK (Selective ACK)** para solicitar apenas os “buracos” na sequência

TCP: Exemplo Gráfico de Seq e ACK (com Perda)

Dinâmica de envio, perda de pacote, ACK duplicado e retransmissão por *Timeout*

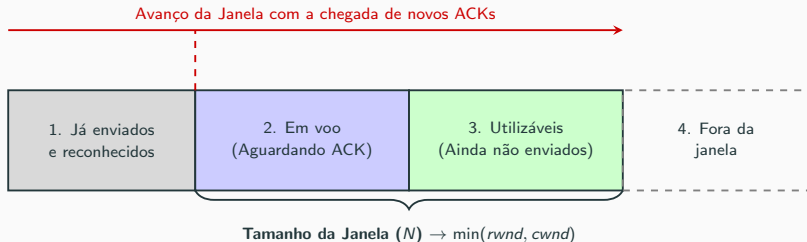


Entendendo a Dinâmica

1. **A** envia 500 bytes a partir do `Seq=1000`
2. **B** recebe com sucesso e responde com `ACK=1500` (espera o byte 1500)
3. **A** envia mais 300 bytes (`Seq=1500`), mas o pacote é **perdido** na rede
4. Sem saber da perda, **A** envia o próximo: 1000 bytes (`Seq=1800`)
5. **B** recebe fora de ordem. Como ainda espera o byte 1500, gera um **ACK duplicado** (1500)
6. O temporizador de **A** para o pacote 1500 expira (**Timeout**)
7. **A** retransmite apenas o segmento perdido (`Seq=1500`)
8. **B** agora possui todos os dados em ordem. Envia **ACK cumulativo** pedindo o byte 2800

Espaço de Sequência do Remetente (Janela Deslizante)

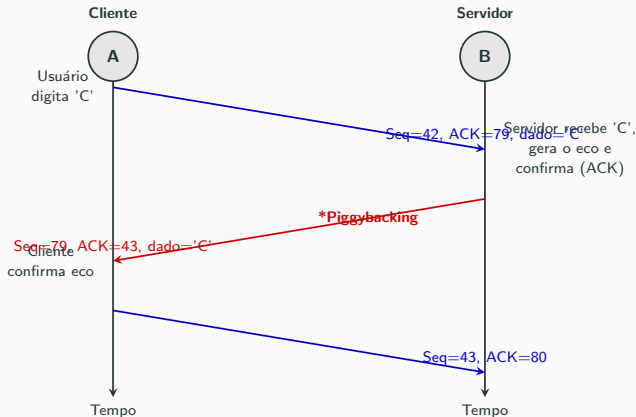
O remetente visualiza o fluxo contínuo de bytes dividido em **quatro categorias** lógicas, que formam a base do mecanismo de Janela Deslizante:



- **A Dinâmica:** Quando um ACK cumulativo chega, a janela “desliza” para a direita, convertendo bytes da Categoria 2 para a 1, e liberando novos bytes da Categoria 4 para a 3
- **O Gargalo:** A soma dos bytes nas Categorias 2 e 3 nunca pode ultrapassar o tamanho da janela (N)

TCP em Ação: O Cenário Telnet (Piggybacking)

Um cenário interativo clássico (*Telnet*) demonstra a evolução contínua dos números de Sequência e ACKs, introduzindo o conceito de **Piggybacking** (pegar carona)



- **A Matemática do ACK:** Como o caractere 'C' possui 1 byte de tamanho, o receptor soma 1 ao Seq recebido para formar o seu ACK ($42 + 1 = 43$)
- **Piggybacking (Carona):** O Servidor não gasta recursos enviando um segmento vazio apenas para o ACK; ele o embute no segmento que carrega a resposta (o eco de 'C')

TCP: Estimativa de RTT e Timeout

O temporizador de retransmissão (*Timeout*) precisa ser dinâmico. Um valor fixo seria ineficiente devido à constante variação de latência e rotas na rede

O Dilema do Tempo de Espera

- **Muito curto:** Causa retransmissões prematuras e desperdício de banda
- **Muito longo:** Causa reação lenta à perda real de pacotes, derrubando a vazão

Amostragem e o Algoritmo de Karn

- **SampleRTT:** Tempo medido entre o envio de um segmento e a chegada do seu ACK
- **Algoritmo de Karn:** O TCP ignora o *SampleRTT* de segmentos retransmitidos. Motivo: Ambiguidade do ACK (não se sabe se o ACK é do envio original atrasado ou da retransmissão)

Estimativa Suavizada (EWMA)

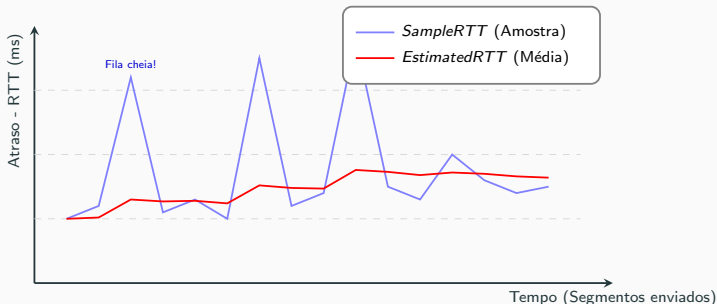
O TCP utiliza uma Média Móvel Ponderada Exponencial para evitar que picos de atraso isolados alterem drasticamente o temporizador:

$$EstimatedRTT = (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT$$

Valor típico recomendado (RFC 6298): $\alpha = 0,125$ (1/8)

Comportamento do RTT: Amostra vs. Estimativa

A diferença visual entre as medições instantâneas e a média suavizada demonstra o impacto do filtro passa-baixas (EWMA) aplicado pelo TCP



Interpretação do Gráfico

- **SampleRTT (Ruído):** Altamente volátil. Os picos ocorrem devido a congestionamentos repentinos e alocação temporária em filas de roteadores
- **EstimatedRTT (Sinal):** Ignora picos isolados (evitando reagir a “falsos alarmes” de congestionamento contínuo), mas acompanha suavemente as mudanças reais de tendência da rede

TCP: Intervalo de Timeout Final

Usar apenas a média (*EstimatedRTT*) não é seguro. Se a rede apresentar alta instabilidade (*jitter*), a variação em torno da média causará constantes falsos positivos de *timeout*

Calculando a Margem de Segurança (DevRTT)

O TCP calcula o desvio absoluto médio para medir o quão “nervosa” a rede está, aplicando a mesma lógica de suavização (EWMA):

$$DevRTT = (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$$

Valor típico (RFC 6298): $\beta = 0,25$ (1/4)

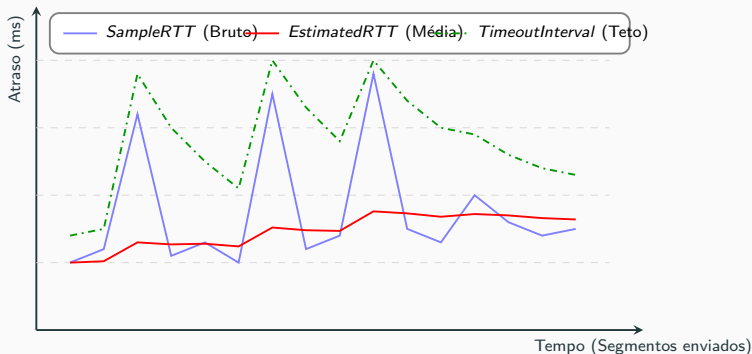
O Temporizador Final (TimeoutInterval)

O valor final soma a média a uma margem de segurança agressiva (4 vezes o desvio) para evitar retransmissões prematuras:

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

Recuo Exponencial (Exponential Backoff): Se ocorrer um *timeout* real (sem ACKs), o TCP **dobra** o valor atual do *TimeoutInterval*. O cálculo pelas fórmulas só é retomado quando novos ACKs chegam

Comportamento do RTT: Integração das Curvas



- **SampleRTT (Ruído):** Altamente volátil, reflete o *jitter* real e momentâneo da rede
- **EstimatedRTT (Filtro):** Acompanha a tendência principal de forma suave, ignorando picos isolados
- **TimeoutInterval (Teto Dinâmico):** Posiciona-se acima da média ($+4 \times DevRTT$). Em momentos de instabilidade, a margem cresce rapidamente para evitar falsas retransmissões

TCP: Transferência Confiável de Dados (rdt)

O TCP cria um duto confiável (*rdt*) operando sobre o serviço de melhor esforço (*best-effort*) e não confiável do protocolo IP

1. Pilares da Confiabilidade:

- **Pipelining:** Envio de múltiplos segmentos em trânsito para maximizar o *throughput*
- **ACKs Cumulativos:** Confirmações em lote que garantem resiliência a perdas isoladas
- **Temporizador Único:** Focado apenas no segmento não reconhecido **mais antigo**, reduzindo o *overhead* no sistema operacional

2. Gatilhos para Retransmissão:

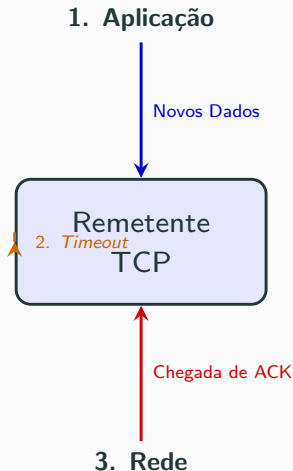
- **Timeout:** Expiração do temporizador principal
- **3 ACKs Duplicados:** Aciona o mecanismo de *Retransmissão Rápida* (*Fast Retransmit*)

Modelagem Simplificada Inicial

Para analisar a máquina de estados, nosso modelo inicial considerará apenas retransmissões por *timeout*, ignorando temporariamente ACKs duplicados e controles de fluxo/congestionamento

Máquina de Estados: Eventos do Remetente TCP

O núcleo do remetente TCP reage essencialmente a três gatilhos. O controle de retransmissão é gerenciado por um **único temporizador** lógico



1. Dados recebidos da Aplicação

- Encapsula dados em segmentos com *Seq Number*
- **Ação do Timer:** Inicia o temporizador (se já não estiver rodando para o pacote não reconhecido mais antigo)

2. Estouro do Temporizador (Timeout)

- Retransmite **apenas** o segmento que causou o *timeout* (reduzindo impacto na rede)
- **Ação do Timer:** Reinicia o temporizador

3. Recepção de ACK Cumulativo

- Se reconhece novos dados, atualiza a base da janela (*SendBase*)
- **Ação do Timer:** Reinicia se houver pacotes pendentes; desliga se todos foram confirmados

Remetente TCP (Simplificado)

A lógica do remetente pode ser resumida em uma estrutura de espera por eventos:

Estrutura do Algoritmo

```
NextSeqNum = InitialSeqNum
```

```
SendBase = InitialSeqNum
```

dados recebidos da aplicação:

```
    criar segmento com seq = NextSeqNum
```

```
    NextSeqNum += length(data)
```

```
    if (timer não rodando) iniciar timer
```

timeout:

```
    retransmitir segmento não confirmado com menor seq
```

```
    iniciar timer
```

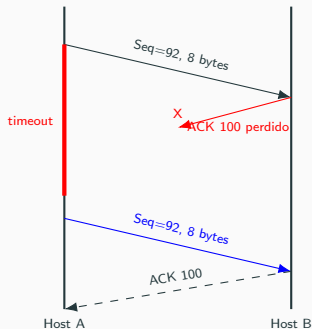
ACK recebido com valor y :

```
    if ( $y > SendBase$ ) {
```

```
        SendBase =  $y$ 
```

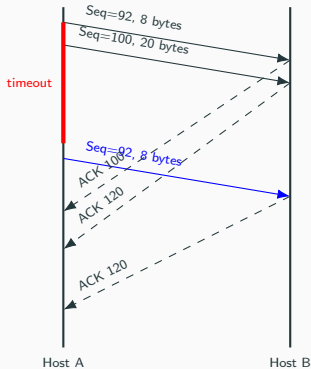
```
        if (existem segmentos não confirmados) iniciar timer
```

TCP: Retransmissão por ACK Perdido



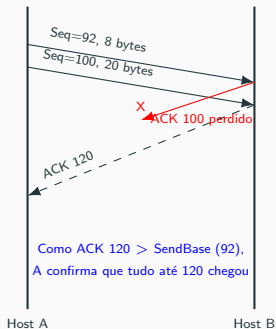
Quando um ACK é perdido, o remetente retransmite o segmento após o *timeout*

TCP: Timeout Prematuro



Se o *timeout* expira antes da chegada do ACK, o remetente retransmite, gerando duplicatas

TCP: Vantagem do ACK Cumulativo



O ACK cumulativo permite que a perda de um ACK anterior seja ignorada se um ACK posterior chegar a tempo

TCP: Regras de Geração de ACKs (RFC 1122/2581)

O receptor TCP segue regras específicas para otimizar o uso da rede

Evento no Receptor	Ação do Receptor TCP
Chegada de segmento em ordem com seq esperado. Tudo anterior já confirmado	ACK atrasado. Espera até 500ms pelo próximo segmento. Se não vier, envia ACK
Chegada de segmento em ordem. Outro segmento aguardando ACK	ACK imediato. Envia um único ACK cumulativo para ambos os segmentos
Chegada de segmento fora de ordem (maior que esperado). Lacuna detectada	ACK duplicado imediato. Indica o número de sequência do próximo byte esperado
Chegada de segmento que preenche lacuna parcial ou total	ACK imediato, desde que comece no limite inferior da lacuna

TCP: Retransmissão Rápida (Fast Retransmit)

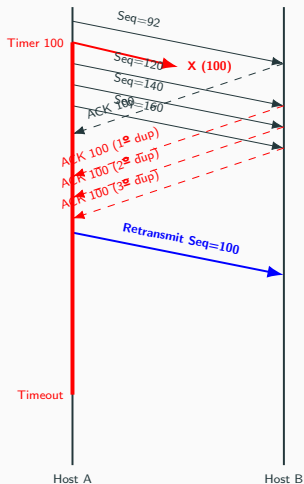
O período de *timeout* do TCP costuma ser longo, o que atrasa a recuperação de pacotes perdidos. A retransmissão rápida agiliza esse processo usando ACKs duplicados

- **A lógica:** Se um segmento for perdido, o receptor enviará ACKs duplicados para cada segmento subsequente que chegar
- **Triplo ACK Duplicado:** Se o remetente receber 3 ACKs extras (total de 4 ACKs) para o mesmo dado:
 - Ele assume que o segmento seguinte foi perdido;
 - Reenvia o segmento com o menor número de sequência pendente **antes** do timer expirar

Por que 3 ACKs?

Pequenas reordenações de rede podem gerar 1 ou 2 ACKs duplicados. O terceiro ACK é um forte indicativo de perda real

Exemplo: Retransmissão Rápida

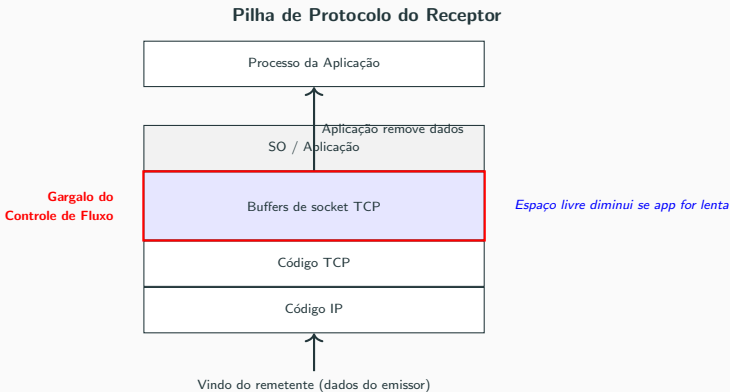


Como Não Sobrecarregar o Receptor: Controle de Fluxo

Controle de Fluxo do TCP

O controle de fluxo garante que o remetente não transbordará o buffer do receptor, adequando a velocidade de transmissão à velocidade de leitura da aplicação

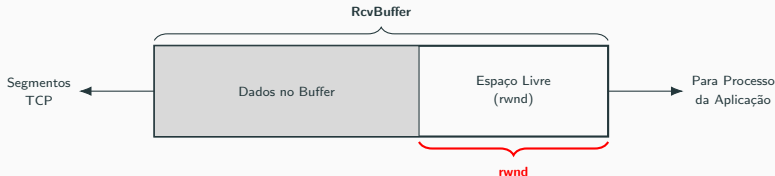
- **Problema:** A aplicação pode remover dados do buffer mais devagar do que o TCP os entrega
- **Solução:** O receptor controla o remetente “anunciando” o espaço disponível no buffer



Controle de Fluxo: A Janela de Recepção (rwnd)

O objetivo é garantir que o remetente não transborde o buffer do receptor

- **Problema:** A aplicação pode ler os dados do buffer mais devagar do que o TCP os entrega
- **Solução:** O receptor controla o remetente “anunciando” o espaço disponível (rwnd)



O receptor comunica o valor de rwnd em cada segmento enviado ao remetente

Mecanismo da Janela de Recepção ($rwnd$)

O remetente limita a quantidade de dados “no ar” ao valor de $rwnd$ anunciado pelo receptor

- O receptor inclui o valor de $rwnd$ no cabeçalho de cada segmento enviado ao remetente
- **Cálculo no Receptor:**

$$rwnd = RcvBuffer - [Dados na fila]$$

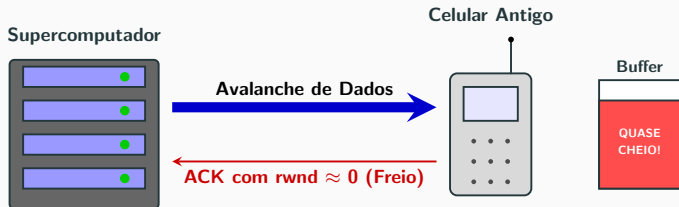
- **Ação do Remetente:** Garante que $(LastByteSent - LastByteAcked) \leq rwnd$

Buffer Cheio

Se $rwnd = 0$, o remetente continua enviando segmentos de 1 byte periodicamente para forçar uma atualização do receptor e evitar o travamento da conexão

Controle de Fluxo: Protegendo o Receptor

O Controle de Fluxo é um serviço **fim a fim** que evita que um transmissor rápido sobrecarregue a memória (*buffer*) de um receptor mais lento



- **A Disparidade:** O servidor envia dados mais rápido do que a aplicação local consegue processar, lotando o *buffer* de recepção
- **A Solução (rwnd):** Em cada ACK, o receptor anuncia a sua **Janela de Recepção (rwnd)**, indicando os bytes de espaço livre restantes
- **O Freio:** Se o *buffer* lotar, ele envia $rwnd = 0$. O remetente suspende o envio e manda apenas pacotes de sondagem (1 byte) até que o receptor tenha espaço

Como Estabelecer Conversas: Gerenciamento de Conexão

Concordando em estabelecer uma conexão

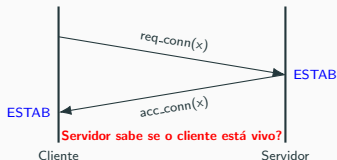
Antes da troca de dados, o emissor e o receptor devem realizar um *handshake* para inicializar variáveis (números de sequência, buffers, etc)

Handshake de 2 vias:

- O cliente envia um pedido ($\text{req_conn}(x)$) e o servidor aceita ($\text{acc_conn}(x)$)

Por que o modelo de 2 vias falha?

- **Atrasos e perdas:** Mensagens retransmitidas podem chegar atrasadas
- **Invisibilidade:** O servidor não tem certeza se o cliente recebeu o aceite
- **Half-open connection:** O servidor aloca recursos para um pedido antigo de um cliente que já não existe



Falha das 2 Vias: O Problema do Atraso

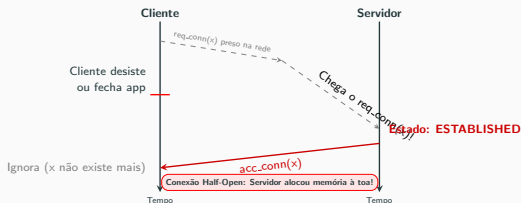
O grande problema do *handshake* de 2 vias é a alocação prematura de estado. O servidor não consegue distinguir entre conexão nova e um **segmento duplicado atrasado** de uma conexão antiga

Causa Raiz: Confiança Cega no Passado

O servidor aloca recursos baseando-se apenas na chegada de um pedido, sem uma terceira via para que o cliente confirme: *“Sim, eu ainda quero essa conexão”*

Isso gera dois cenários de desperdício de recursos no servidor:

- **Conexões Duplicadas:** O cliente tentou de novo, o servidor aceita a nova e, mais tarde, aceita o pedido original atrasado
- **Conexões Meio-Abertas (Half-Open):** O servidor aceita um pedido fantasma após o cliente já ter desistido



TCP: Handshake de 3 vias

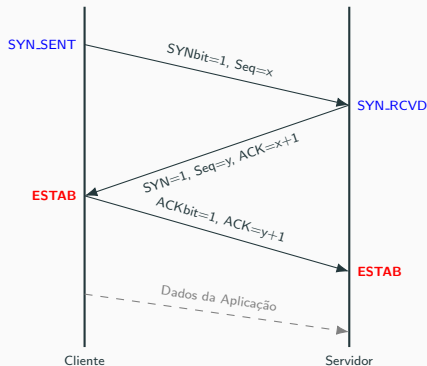
O protocolo de três vias garante que o emissor e o receptor sincronizem seus números de sequência iniciais e confirmem a disponibilidade mútua

A Lógica da Troca:

- **SYN**: Cliente escolhe seq x e solicita conexão
- **SYN-ACK**: Servidor escolhe seq y , confirma x ($ACK=x+1$) e envia seu SYN
- **ACK**: Cliente confirma y ($ACK=y+1$)

Vivacidade

O servidor só aloca recursos definitivos (estado **ESTAB**) após o terceiro passo, evitando conexões falsas



TCP: Fechando uma Conexão

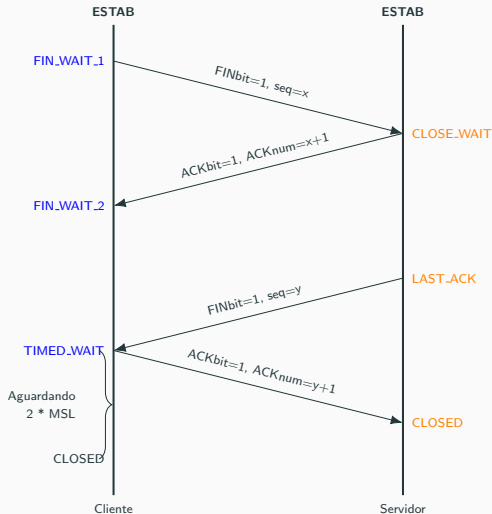
No TCP, o encerramento da conexão é feito de forma independente por cada host (cliente e servidor)

- **O Bit FIN:** Para fechar sua parte da conexão, o host envia um segmento com o bit de controle **FIN = 1**
- **Confirmação:** O outro lado deve responder ao FIN recebido com um **ACK**
- **Simultaneidade:** O protocolo é robusto o suficiente para tratar situações onde ambos decidem fechar a conexão ao mesmo tempo

Características Importantes

- Ao receber um FIN, o ACK pode ser enviado sozinho ou “carregado” junto com o próprio FIN do receptor (caso ele também queira fechar imediatamente)
- Após o envio do FIN, o host entra em um estado onde não pode mais enviar dados, mas ainda pode recebê-los

Diagrama de Fechamento de Conexão



Resumo: A Essência do TCP

O TCP transforma o serviço de “melhor esforço” do IP em um canal de comunicação robusto, garantindo que os dados cheguem íntegros e em ordem

1. Transferência Confiável

- **Numeração:** Sequenciamento por bytes
- **Recuperação:** Retransmissão por *Timeout* ou **Rápida** (3 ACKs duplicados)

2. Temporização Adaptativa

- Cálculo dinâmico de RTT para evitar retransmissões precoces

3. Controle de Fluxo

- Uso da janela **rwnd** para respeitar o limite do receptor

4. Gerência de Conexão

- **Abertura:** Handshake de 3 vias
- **Fechamento:** Handshake de 4 vias (FIN)

